



APOSTILA · CURSO DE FRONT-END · MÓDULO VUE.JS

Vue.js — Aula 2

Reatividade, Formulários e Listas

Eventos com v-on · v-model · v-for e :key · methods x computed

Material do aluno · Inclui 10 exercícios

Bem-vindo à Aula 2

Na Aula 1 aprendemos a **exibir** dados na tela. Hoje a tela deixa de ser só leitura: o usuário vai **interagir** (clicar, digitar, marcar opções) e os dados vão responder em tempo real.

O que já sabemos (Aula 1)	Para que serve
<code>data()</code>	Guarda os dados da aplicação
<code>{{ }}</code>	Mostra um dado como texto no HTML
<code>v-bind</code> ou <code>:</code>	Liga um dado a um atributo da tag
<code>v-if</code> / <code>v-show</code>	Mostra ou esconde um elemento

Como montar os arquivos

Para testar os exemplos e fazer os exercícios, crie uma pasta com dois arquivos: `index.html` e `app.js`.

```
<!-- index.html -->
<div id="app">
  <!-- seu código Vue vai aqui -->
</div>

<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<script src="app.js"></script>
```

```
// app.js
const { createApp } = Vue;

createApp({
  data() {
    return { };
  }
}).mount("#app");
```

PARTE 1

v-on: reagindo a cliques

O `v-on` faz alguma coisa acontecer quando o usuário clica (ou digita, ou envia um formulário).

```
// app.js
createApp({
  data() {
    return { contador: 0 };
  },
  methods: {
    incrementar() {
      this.contador++;
    }
  }
}).mount("#app");
```

```
<!-- index.html -->
<p>{{ contador }}</p>
<button v-on:click="incrementar">+1</button>

<!-- forma curta (a mais usada) -->
<button @click="incrementar">+1</button>
```

Guarde isto

- As funções ficam dentro de `methods`.
- `@click` é o mesmo que `v-on:click`, só que mais curto.
- Dentro de `methods` usamos `this` para acessar os dados: `this.contador++`.

PARTE 2

v-model: ligando o formulário aos dados

O `v-model` liga um campo do formulário a um dado. Mudou um, muda o outro.

```
// app.js
data() {
  return { nome: "" };
}
```

```
<!-- index.html -->
<input v-model="nome" placeholder="Seu nome">
<p>Olá, {{ nome }}!</p>
```

Digite no campo e veja o texto mudar na hora. O dado `nome` é sempre igual ao que está escrito no campo.

Checkbox e select

O `v-model` também funciona em outros elementos. O que muda é o **tipo** do dado:

```
// app.js
data() {
  return {
    aceito: false,      // checkbox -> true ou false
    corFavorita: "verde" // select   -> texto da opção
  };
}
```

```
<!-- index.html -->
<input type="checkbox" v-model="aceito">
<p>Aceitou: {{ aceito }}</p>

<select v-model="corFavorita">
  <option value="verde">Verde</option>
  <option value="azul">Azul</option>
</select>
```

PARTE 3

v-for: mostrando listas

O **v-for** repete um elemento para cada item de um array.

```
// app.js
data() {
  return {
    tarefas: ["Estudar Vue", "Fazer exercício", "Revisar HTML"]
  };
}
```

```
<!-- index.html -->
<ul>
  <li v-for="tarefa in tarefas" :key="tarefa">
    {{ tarefa }}
  </li>
</ul>

<!-- com o número da posição -->
<li v-for="(tarefa, index) in tarefas" :key="tarefa">
  {{ index + 1 }} - {{ tarefa }}
</li>
```

Leia **tarefa in tarefas** assim: "para cada **tarefa** dentro de **tarefas**".

Para que serve o :key?

O **:key** é uma "etiqueta" que identifica cada item da lista. Com ela, o Vue não se confunde quando itens são adicionados, removidos ou mudam de lugar.

Regra prática

Use sempre um valor **único** no **:key**. Se o item tiver um **id**, use **:key="item.id"**. Evite usar o índice quando a lista pode mudar.

PARTE 4

methods x computed

As duas servem para calcular coisas, mas funcionam de jeitos diferentes.

	methods	computed
Quando roda	Toda vez que é chamado	Só quando os dados que ele usa mudam (guarda o resultado)
No HTML	Com parênteses: <code>{{ calcular() }}</code>	Sem parênteses: <code>{{ restantes }}</code>
Use para	Ações: cliques, adicionar, remover	Valores mostrados na tela: totais, contagens

Exemplo de computed

```
// app.js
createApp({
  data() {
    return {
      tarefas: [
        { texto: "Estudar Vue",      feita: true },
        { texto: "Fazer exercício",  feita: false },
        { texto: "Revisar HTML",     feita: false }
      ]
    };
  },
  computed: {
    restantes() {
      return this.tarefas.filter(t => !t.feita).length;
    }
  }
}).mount("#app");
```

```
<!-- index.html -->
<p>{{ restantes }} tarefa(s) restante(s)</p>
```

Resumo da aula

Recurso	Em uma frase
@click	Chama um método quando o usuário clica
v-model	Liga um campo do formulário a um dado
v-for	Repete um elemento para cada item de uma lista
:key	Dá uma identidade única a cada item da lista
methods	Funções para ações; rodam sempre que chamadas
computed	Valores calculados; só recalculam quando precisam

Exercícios

Faça cada exercício em uma pasta separada, com `index.html` e `app.js`. Marque os quadradinhos conforme for concluindo cada item.

Exercício 1 — Contador

FÁCIL · @click, methods

Crie um contador que começa em 0.

- ☐ Um botão **+1** que soma 1.
- ☐ Um botão **-1** que subtrai 1.
- ☐ Um botão **Zerar** que volta para 0.

Exercício 2 — Olá, visitante

FÁCIL · v-model, v-if

Crie um campo onde o usuário digita o nome.

- ☐ Ligue o campo ao dado `nome` com `v-model`.
- ☐ Mostre a frase "Olá, {nome}!" abaixo do campo.
- ☐ A frase só deve aparecer se o nome não estiver vazio (use `v-if`).

Exercício 3 — Nome completo

FÁCIL · v-model, computed

Crie dois campos: **nome** e **sobrenome**.

- ☐ Crie um `computed` chamado `nomeCompleto` que junta os dois com um espaço.
- ☐ Mostre o nome completo na tela.

Exercício 4 — Minhas preferências

FÁCIL · v-model em checkbox e select

Crie um pequeno formulário de preferências.

- ☐ Um checkbox "Quero receber novidades". Mostre na tela `true` ou `false`.
- ☐ Um select com 3 cores. Mostre "Sua cor favorita é: {cor}".
- ☐ Faça a cor escolhida já vir marcada ao abrir a página.

Exercício 5 — Lista de frutas

MÉDIO · v-for, índice

Crie um array com 5 frutas e mostre-as em uma lista `<ul>`.

- ☐ Use `v-for` para mostrar cada fruta.
- ☐ Mostre a posição antes do nome: "1 - Maçã", "2 - Banana"...
- ☐ Não esqueça do `:key`.

Exercício 6 — Adicionando frutas

MÉDIO · v-model, @click, push

Continue o exercício 5.

- ☐ Adicione um campo e um botão **Adicionar**.
- ☐ Ao clicar, a fruta digitada entra no final da lista (**push**).
- ☐ Depois de adicionar, o campo deve ficar vazio.
- ☐ Extra: não deixe adicionar se o campo estiver vazio.

Exercício 7 — Lista de alunos

MÉDIO · v-for com objetos, :key

Crie um array de 4 alunos no formato { **id**, **nome**, **nota** }.

- ☐ Mostre cada aluno assim: "Ana — nota 8".
- ☐ Use **:key="aluno.id"**.
- ☐ Crie um **computed** que calcula a **média** das notas e mostre na tela.

Exercício 8 — Quem passou?

MÉDIO · computed

Continue o exercício 7.

- ☐ Crie um **computed** **aprovados** que conta quantos alunos têm nota maior ou igual a 7.
- ☐ Mostre: "{aprovados} de {total} alunos aprovados".
- ☐ Mude as notas no **data()** e veja o resultado atualizar.

Exercício 9 — Pense e responda

MÉDIO · methods x computed, :key

Responda com suas palavras:

- ☐ a) Qual a diferença entre **methods** e **computed**?
- ☐ b) Para mostrar o total de itens de uma lista na tela, você usaria qual dos dois? Por quê?
- ☐ c) Por que é melhor usar um **id** no **:key** em vez do índice?

Exercício 10 — Lista de Tarefas (To-Do List)

DESAFIO · tudo da aula

Junte tudo o que você aprendeu em uma lista de tarefas.

- ☐ Um campo com `v-model` para digitar a nova tarefa.
- ☐ Um botão que adiciona a tarefa ao array (o `v-for` mostra a lista).
- ☐ Cada tarefa tem um checkbox com `v-model` para marcar como feita.
- ☐ Um `computed` mostrando quantas tarefas faltam.
- ☐ Cada `<li>` com `:key` único (use um `id`, não o índice).

```
// Dica de estrutura
data() {
  return { novaTarefa: "", tarefas: [] };
},
methods: {
  adicionar() {
    this.tarefas.push({
      id: Date.now(),
      texto: this.novaTarefa,
      feita: false
    });
    this.novaTarefa = "";
  }
}
```

Próxima aula

Componentização: `props`, `emits` e comunicação entre componentes.